

QUESTIONS

Q1

Scalable Smart Building Management System
10 marks

✓

Q2

Smart Renewable Energy and EV Charging System
10 marks

✓

2/2 answered

Smart Renewable Energy and EV Charging System

10 marks

A smart city is developing a Renewable Energy and EV Charging Management System integrating solar power, battery storage, and EV charging stations. Current issues include uneven energy distribution, difficulty managing energy sources, flexible charging cost calculation, difficulty supporting future renewable technologies, and inefficient dynamic handling. Develop a scalable C++ OOP solution using inheritance, multiple inheritance, virtual base classes, abstract classes, constructors and destructors, composition, pointers, file points, pointer to derived classes, and runtime polymorphism. Design abstract EnergySource with SolarEnergy, WindEnergy, and BatteryStorage. Create HybridChargingStation using multiple inheritance, use a virtual base class, and include a ChargingMonitor member class. Implement pure virtual calculateEnergyCost(). SolarEnergy uses solar energy ♦ charging rate, WindEnergy uses wind energy ♦ charging rate, BatteryStorage uses stored energy ♦ discharge rate. Demonstrate EnergySource*, new, delete, pointer to derived class, this pointer, polymorphism, and a menu-driven program.

Your Code

```
#include <iostream>
using namespace std;

class EnergySource {
public:
    EnergySource() {}
    virtual double calculateEnergyCost() = 0;
    virtual ~EnergySource() {}
};

class SolarEnergy : virtual public EnergySource {
    double energy, rate;
public:
    SolarEnergy(double s = 0, double r = 0) {
        energy = s;
        rate = r;
    }
    double calculateEnergyCost() {
        return energy * rate;
    }
    void setValues(double s, double r) {
        this->energy = s;
        this->rate = r;
    }
};

class WindEnergy : virtual public EnergySource {
    double energy, rate;
public:
    WindEnergy(double w = 0, double r = 0) {
        energy = w;
        rate = r;
    }
    double calculateEnergyCost() {
        return energy * rate;
    }
    void setValues(double w, double r) {
        this->energy = w;
        this->rate = r;
    }
};

class BatteryStorage : virtual public EnergySource {
    double energy, rate;
public:
    BatteryStorage(double b = 0, double r = 0) {
        energy = b;
        rate = r;
    }
    double calculateEnergyCost() {
        return energy * rate;
    }
    void setValues(double b, double r) {
        this->energy = b;
        this->rate = r;
    }
};

class ChargingMonitor {
public:
    void display(double s, double w, double b) {
        cout << "Solar Cost: " << s << endl;
        cout << "Wind Cost: " << w << endl;
        cout << "Battery Cost: " << b << endl;
    }
};

class HybridChargingStation { public SolarEnergy,
                             public WindEnergy,
                             public BatteryStorage {
    ChargingMonitor monitor;
public:
    HybridChargingStation(double ss, double ww,
                          double bw, double wr,
                          double bs, double br)
        : SolarEnergy(ss, sr),
          WindEnergy(ww, wr),
          BatteryStorage(bs, br) {}
    double calculateEnergyCost() {
        return SolarEnergy::calculateEnergyCost()
            + WindEnergy::calculateEnergyCost()
            + BatteryStorage::calculateEnergyCost();
    }
    void display() {
        monitor.display();
        SolarEnergy::calculateEnergyCost();
        WindEnergy::calculateEnergyCost();
        BatteryStorage::calculateEnergyCost();
    }
    cout << "Total Cost: "
        << calculateEnergyCost() << endl;
}

int main() {
    double ss, sr, ww, wr, bw, br;
    cin >> ss >> sr;
    cin >> ww >> wr;
    cin >> bw >> br;
    HybridChargingStation *p =
        new HybridChargingStation(ss, sr, ww, wr, bw, br);
    EnergySource *base = p;
    int choice;
    do {
        cin >> choice;
        switch (choice) {
            case 1:
                p->display();
                break;
            case 2:
                cin >> ss >> sr;
                p->SolarEnergy::setValues(ss, sr);
                break;
            case 3:
                cin >> ww >> wr;
                p->WindEnergy::setValues(ww, wr);
                break;
            case 4:
                cin >> bw >> br;
                p->BatteryStorage::setValues(bw, br);
                break;
            case 5:
                cout << "Total Cost: "
                    << base->calculateEnergyCost() << endl;
                break;
            case 6:
                break;
            default:
                cout << "Invalid Choice" << endl;
        }
    } while (choice != 0);
    delete p;
    return 0;
}
```

Input

```
100 2
80 3
200 1
1
```

Output

QUESTIONS

- Q1

Calculate Smart System Management System
30 marks

✓
- Q2

Smart Renewable Energy and EV Charging System
50 marks

✓

2/2 answered

Q2 Smart Renewable Energy and EV Charging System

50 marks

A smart city is developing a Renewable Energy and EV Charging Management System, integrating solar power, battery storage, and EV charging stations. Current issues include uneven energy distribution, difficulty managing energy sources, inflexible charging/cost calculation, difficulty supporting future renewable technologies, and inefficient demand handling. Develop a scalable C++ OOP solution using inheritance, multiple inheritance, virtual base classes, abstract classes, constructors and destructors, composition, pointers, this pointer, pointer to derived classes, and runtime polymorphism. Design abstract EnergySource with SolarEnergy, WindEnergy, and BatteryStorage. Create HybridChargingStation using multiple inheritance, use a virtual base class, and include a ChargingMonitor member class. Implement pure virtual calculateEnergyCost(). SolarEnergy uses solar energy ♦ charging rate, WindEnergy uses wind energy ♦ charging rate, BatteryStorage uses stored energy ♦ discharge rate. Demonstrate EnergySource*, new, delete, pointer to derived class, this pointer, polymorphism, and a menu-driven program.

Your Code

```
#include <iostream>
using namespace std;

class EnergySource {
public:
    EnergySource() {}
    virtual double calculateEnergyCost() = 0;
    virtual ~EnergySource() {}
};

class SolarEnergy : virtual public EnergySource {
    double energy, rate;

public:
    SolarEnergy(double s = 0, double r = 0) {
        energy = s;
        rate = r;
    }

    double calculateEnergyCost() {
        return energy * rate;
    }

    void setData(double s, double r) {
        this->energy = s;
        this->rate = r;
    }
};

class WindEnergy : virtual public EnergySource {
    double energy, rate;

public:
    WindEnergy(double s = 0, double r = 0) {
        energy = s;
        rate = r;
    }

    double calculateEnergyCost() {
        return energy * rate;
    }

    void setData(double s, double r) {
        this->energy = s;
        this->rate = r;
    }
};

class BatteryStorage : virtual public EnergySource {
    double energy, rate;

public:
    BatteryStorage(double s = 0, double r = 0) {
        energy = s;
        rate = r;
    }

    double calculateEnergyCost() {
        return energy * rate;
    }

    void setData(double s, double r) {
        this->energy = s;
        this->rate = r;
    }
};

class HybridChargingStation : public SolarEnergy,
                             public WindEnergy,
                             public BatteryStorage {
    ChargingMonitor monitor;

public:
    HybridChargingStation(double sr, double wr,
                          double bs, double br) {
        SolarEnergy(sr, wr);
        WindEnergy(sr, wr);
        BatteryStorage(sr, br);
    }

    double calculateEnergyCost() {
        return SolarEnergy::calculateEnergyCost() +
               WindEnergy::calculateEnergyCost() +
               BatteryStorage::calculateEnergyCost();
    }

    void display() {
        cout << "Solar Energy: " << SolarEnergy::calculateEnergyCost() << endl;
        cout << "Wind Energy: " << WindEnergy::calculateEnergyCost() << endl;
        cout << "Battery Storage: " << BatteryStorage::calculateEnergyCost() << endl;
        cout << "Total Cost: " << calculateEnergyCost() << endl;
    }
};

int main() {
    double sr, wr, br, bs, hr;

    cout << "Enter sr, wr, br, bs, hr: ";
    cin >> sr >> wr >> br >> bs >> hr;

    HybridChargingStation hc =
        new HybridChargingStation(sr, wr, br, bs, hr);

    EnergySource *base = hc;

    int choice;

    do {
        cout << "Choice: ";
        switch (choice) {
            case 1:
                hc->display();
                break;

            case 2:
                hc->SolarEnergy::setData(sr, wr);
                break;

            case 3:
                hc->WindEnergy::setData(wr, wr);
                break;

            case 4:
                hc->BatteryStorage::setData(br, br);
                break;

            case 5:
                cout << "Total Cost: "
                    << base->calculateEnergyCost() << endl;
                break;

            case 6:
                break;

            default:
                cout << "Invalid Choice" << endl;
        }
    } while (choice != 6);

    delete hc;

    return 0;
}
```

Input

100 2
50 3
200 1
1
-

Output

Visible Test Cases